

Workload-Aware KV-Cache Management for LLM Inference

Waseem Ahmad
Harvard University

Abstract

Large Language Models (LLMs) rely on Key-Value (KV) caches during inference to avoid recomputing attention states for previously processed tokens. As sequence lengths and concurrency grow, KV-cache memory becomes a primary systems bottleneck, especially in long-context and multi-user serving scenarios. Although many deployments use simple eviction policies such as Least Recently Used (LRU), real LLM access patterns are workload-dependent and often extend beyond purely recency-based reuse.

In this project, we develop a trace-driven, block-level KV-cache simulator for controlled and reproducible evaluation under constrained memory budgets. We benchmark FIFO and LRU across short-context, multi-turn, and long-context synthetic workloads, then extend the simulator to a MOONCAKE-inspired shared-global cache that enables cross-request reuse. Finally, we introduce an adaptive shared-global policy that responds online to workload shifts and memory pressure using recency, frequency, recomputation cost, and shared-prefix signals.

Across 15-seed evaluations, we observe that LRU consistently outperforms FIFO, with larger gains in more difficult workloads. More importantly, shared-global caching provides the largest system-level improvement by increasing hit rate and reducing recomputation relative to isolated per-request caching. Adaptive eviction yields additional robustness under non-stationary workloads, with modest hit-rate gains and consistent recomputation savings. Overall, our results frame KV-cache management as a storage systems problem and show that workload-aware and shared caching designs can materially improve the efficiency of modern LLM serving systems.

1 Introduction

Large Language Models (LLMs) have rapidly become one of the dominant workloads in modern computing systems. As models continue to scale in both parameter count and

context length, inference efficiency has become increasingly important for reducing latency and serving cost. One of the key optimizations used during inference is the Key-Value (KV) cache, which stores intermediate attention states for previously processed tokens so they can be reused instead of recomputed during future decoding steps.

While KV-caching significantly reduces computation, it introduces a new systems-level bottleneck, which is that memory usage can grow approximately linearly with sequence length. In workloads such as long document processing, retrieval-augmented generation, or multi-turn conversations, KV-cache memory can quickly become extremely large. Since serving systems operate under limited GPU memory and often handle many concurrent users, they cannot retain every cached token indefinitely, and, as a result, eviction decisions become critical for balancing memory usage, recomputation cost, and overall performance.

Many current systems rely on relatively simple eviction strategies such as First-In First-Out (FIFO) or Least Recently Used (LRU). However, LLM workloads exhibit highly diverse access patterns, so these policies may not always be the most relevant in every single scenario. For example, short prompts often demonstrate strong temporal locality (i.e., blocks that were accessed recently are likely to be accessed again soon), while long-context workloads may require reusing information from much earlier in the sequence. Multi-turn conversations additionally introduce repeated prefix reuse through shared system prompts and prior conversational context. These behaviors suggest that workload-aware caching strategies may outperform static, one-size-fits-all policies.

Recent systems research, such as MOONCAKE, has further motivated treating KV-cache as a shared systems resource rather than an isolated per-request optimization [4]. By enabling reuse across requests, shared caching can significantly reduce redundant computation and improve serving efficiency. Inspired by this direction, this project studies KV-cache management from a storage systems perspective and explores how workload characteristics influence cache behavior under constrained memory settings.

To study these tradeoffs, we build a trace-driven KV-cache simulator that models LLM-style access patterns at the block level. The simulator supports multiple workload families, including short-context, long-context, and multi-turn conversational workloads, and evaluates different eviction strategies using metrics such as cache hit rate and recomputation cost. We additionally extend the simulator with shared-global caching and an adaptive eviction controller that dynamically responds to workload shifts and memory pressure.

Our contributions are as follows:

- We design and implement a trace-driven, block-level KV-cache simulator for studying workload-aware caching behavior in LLM inference systems, with reproducible multi-seed evaluation and interpretable metrics.
- For the **75% stage**, we establish baseline policy behavior by evaluating FIFO and LRU across short-context, multi-turn, and long-context workloads, including cache-size sweeps to analyze scaling under different memory capacities.
- For the **100% stage**, we extend the simulator to a MOONCAKE-inspired shared-global cache and demonstrate that cross-request prefix reuse improves hit rate and reduces recomputation cost compared to isolated per-request caching under matched memory budgets.
- For the **125% stage**, we add an adaptive shared-global eviction strategy and show improved robustness under workload shifts and changing memory pressure through phase-aware and overall evaluation.

2 Background and Motivation

2.1 KV-Cache in LLM Inference

During inference, LLMs generate one token at a time. To avoid recomputing attention over all previous tokens at every step, serving systems store intermediate key and value states in a KV-cache. This cache improves performance, but it also grows with sequence length. As prompts become longer and conversations continue over many turns, KV-cache memory can become a major bottleneck. Because cache capacity is limited, a serving system cannot keep every cached block forever. Once the cache is full, it must decide which blocks to evict. A poor eviction decision can remove blocks that are reused later, causing additional recomputation. This makes KV-cache management similar to traditional storage caching problems [?], but with access patterns shaped by LLM workloads.

2.2 Motivation from Shared KV-Cache Systems

Recent systems such as MOONCAKE demonstrate that KV-cache can be treated as a shared storage resource rather than a purely local per-request optimization [4]. Instead of maintaining completely isolated caches for each request, shared KV-cache systems allow multiple concurrent requests to reuse previously computed blocks, particularly when requests contain overlapping prefixes, repeated system prompts, or similar conversational structure. This reduces redundant computation and can significantly improve both latency and overall serving efficiency, especially in high-concurrency environments where many users may share common prompt patterns.

This motivates studying not only the behavior of individual eviction policies, but also how cache sharing itself changes workload dynamics and memory efficiency. In particular, a shared-global cache introduces new tradeoffs between fairness, reuse, and eviction pressure, since requests now compete for a common memory pool while simultaneously benefiting from cross-request reuse opportunities. Inspired by this direction, this project first evaluates baseline local cache behavior under different workload types and then extends the simulator toward shared-global and adaptive caching strategies that dynamically respond to workload shifts and memory pressure.

3 Design and Methodology

3.1 Simulator Architecture

We implemented a Python-based KV-cache simulator that operates at the block level. Each cache entry represents a synthetic token block rather than a full tensor, attention head, or model state. This abstraction keeps the simulator lightweight, computationally inexpensive, and easy to run on a normal laptop, while still capturing the key cache dynamics: accesses, hits, misses, evictions, and recomputation overhead. Compared to running full LLM inference, the simulator gives the ability to perform more rapid experimentation across many workloads, cache capacities, and random seeds, where a seed is simply a fixed initialization value for the workload generator that controls the randomness of synthetic access patterns and ensures experiments can be reproduced consistently across runs, without requiring specialized GPU hardware or large computational resources.

Each block stores metadata including key, creation time, last access time, and access count. In shared-global experiments, blocks additionally carry a recomputation cost and a shared-prefix indicator so the simulator can model asymmetric block value and cross-request reuse behavior. The cache uses a fixed capacity measured in number of blocks, and when the cache becomes full, eviction decisions are delegated to the active policy. We structured the simulator into modular components (cache, policies, workload generators, metrics, and

experiment runners) so each part can be tested independently and extended without redesigning the system.

3.2 Workload Generation

The simulator evaluates three single-request LLM-like workload families, and also includes concurrent shared-cache traces for the later project stages.

Short-context workloads model small prompts with strong temporal locality. These traces use a small working set and repeatedly reuse recently accessed blocks, which makes them relatively cache-friendly and useful for validating baseline behavior.

Long-context workloads model long prompts or document-style generation. These traces use a much larger working set, weaker locality, and occasional long-range reuse. As a result, they create sustained cache pressure and are significantly harder for simple policies.

Multi-turn workloads model chatbot-style sessions. These traces combine reused early/system blocks with recent-turn locality from ongoing dialogue history, producing a mixed locality pattern that sits between short and long-context difficulty.

For the 100% and 125% stages, I extended workload generation to concurrent multi-request event streams with shared prefixes and request alternating. To model non-stationary behavior, I added shifted two-phase workloads. The event stream is split at a fixed transition point (`'split_idx'`) into **Phase 1** and **Phase 2**. Phase 1 is a more stable regime with stronger reuse (higher shared-prefix and recency locality), while Phase 2 increases difficulty by reducing reuse and increasing accesses to newer/less-recent blocks. I evaluate both **moderate** and **hard** shifts, where hard shifts apply a larger change in reuse/locality parameters. This setup allows policies to be evaluated in both steady-state and post-shift conditions.

3.3 Eviction Policies

The baseline eviction policies are FIFO and LRU. FIFO evicts blocks in insertion order, independent of recent reuse. LRU evicts the block that has gone unused for the longest time, so it explicitly favors temporal locality. These two policies provide clear and interpretable baselines for studying how recency sensitivity changes cache performance across different workload structures.

I then extended the simulator to a MOONCAKE-inspired shared-global setting in which multiple requests compete for one centralized cache, rather than each request using an isolated local cache. In the local setting, a reused prefix that appears across requests is duplicated in separate caches and cannot be shared. In the shared-global setting, the same prefix block can be retained once and reused by many requests, improving memory efficiency and reducing repeated recomputation. This extension allows direct comparison between

local-isolated and shared-global modes under matched total memory budgets, making cross-request reuse benefits measurable.

3.4 Adaptive Strategy

The adaptive policy is implemented for the shared-global setting and updates its behavior as workload conditions change. In this case, the policy monitors recent miss behavior (using a short rolling history) as a proxy for cache pressure. When pressure is low, the policy is more permissive. When miss pressure rises, it applies stricter admission control so low-value incoming blocks are less likely to displace useful resident blocks.

Eviction is driven by a composite score that combines recency, frequency, recomputation cost, and shared-prefix importance. Intuitively, blocks that are old, rarely reused, cheap to recompute, and not shared are more evictable, while blocks that are recent, repeatedly reused, expensive to recompute, or shared across requests are favored for retention. The relative weighting of these signals is pressure-aware, meaning that during miss spikes, the policy shifts toward stronger recency sensitivity and more selective retention, while still preserving high-cost and high-reuse blocks when useful.

This design intentionally avoids heavy machinery (e.g., learning-based controllers, reinforcement learning policies, etc.) and keeps the implementation lightweight and interpretable. Even so, it enables meaningful policy adaptation across stable phases and post-shift phases, which is the core requirement for the 125% stage.

3.5 Evaluation Metrics

The primary metric for evaluation is **cache hit rate**, defined as the fraction of accesses served directly from cache. Since misses trigger recomputation, we also report **miss count** and **recomputation cost**. In the baseline (75%) simulator, recomputation cost is unit-weighted, so each miss contributes a cost of 1. In the shared-global and adaptive stages, recomputation cost is block-dependent, so the metric reflects both miss frequency and miss severity (i.e., missing a more expensive block is penalized more).

To make comparisons easier across policies and settings, we report per-run totals and multi-seed aggregates (mean and standard deviation over 15 seeds). This helps separate consistent improvements from seed-specific noise in synthetic traces. Additionally, we report workload-level summary statistics: total accesses, number of unique blocks, reuse ratio, shared-prefix fraction (for concurrent traces), and average accesses per unique block. These summaries provide context for workload difficulty. For example, workloads with larger unique-block footprints and lower reuse ratios are expected to produce lower hit rates and higher recomputation, even with stronger policies.

4 Experimental Setup

All experiments were run locally using our Python simulator and evaluated over 15 deterministic random seeds per setting. Using multiple seeds reduces sensitivity to any single synthetic trace and provides more reliable policy comparisons. For each configuration, we report aggregated statistics (mean and standard deviation) to distinguish consistent trends from random variation.

For the baseline (75%) experiments, we compare FIFO and LRU on short-context, multi-turn, and long-context workloads at a fixed cache capacity. We also run cache-size sweeps to measure how hit rate and recomputation change as memory capacity increases, which helps characterize each policy’s scaling behavior under growing cache budgets.

For the 100% experiments, we compare isolated local caching (per-request caches) against a shared-global cache where concurrent requests compete for one central memory pool. The concurrent traces include shared prefixes and interleaved requests to model cross-request reuse opportunities. We also run sensitivity sweeps over concurrency level and shared-prefix intensity to test whether shared-cache gains remain stable across workload regimes.

For the 125% experiments, we evaluate FIFO, LRU, and adaptive policies on shifted two-phase concurrent workloads. We test multiple capacities and shift severities (moderate and hard), and we report both phase-level and overall metrics. This setup allows us to measure not only steady-state performance but also robustness after workload distribution changes, which is a central part of adaptive cache management.

5 Results

5.1 Baseline FIFO vs. LRU

Figure 1 compares cache hit rate across workload families for FIFO and LRU at capacity 128. Across all workload types, LRU is equal to or better than FIFO, with the largest gains appearing as workload difficulty increases. In short-context traces, both policies perform similarly and achieve very high hit rates because the working set is small and reuse is strong. In multi-turn traces, LRU provides a modest but consistent gain by better preserving recently reused conversational blocks and repeated prefix/system context.

The largest separation appears in long-context workloads, where the unique-block footprint is much larger than cache capacity. In this case, the eviction quality becomes more important, since LRU retains recently reused blocks that still have near-term value, while FIFO may evict useful blocks simply because they were inserted earlier. This explains why long-context traces show lower absolute hit rates overall, but still meaningful relative improvement from recency-aware eviction.

Figure 2 shows the corresponding recomputation cost. As expected, the recomputation trend mirrors miss behavior, where lower hit rate leads directly to higher recomputation overhead. Long-context workloads therefore incur substantially more recomputation than short-context or multi-turn workloads. Hence, the baseline results establish two key points for later stages: (1) policy choice matters more in harder workloads, and (2) miss-heavy long-context scenarios are the main target for system-level improvements beyond local FIFO/LRU.

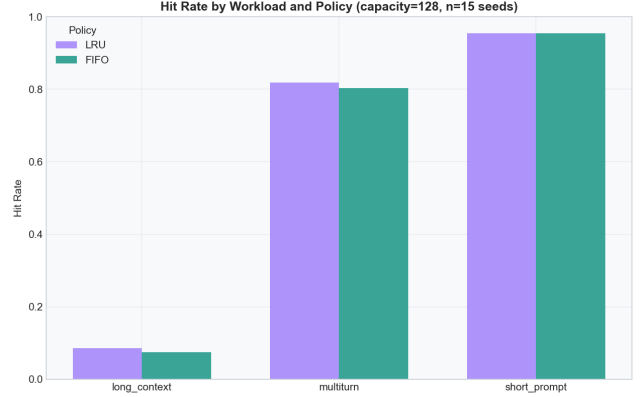


Figure 1: Cache hit rate across workload families for FIFO and LRU at capacity 128, averaged across 15 seeds.

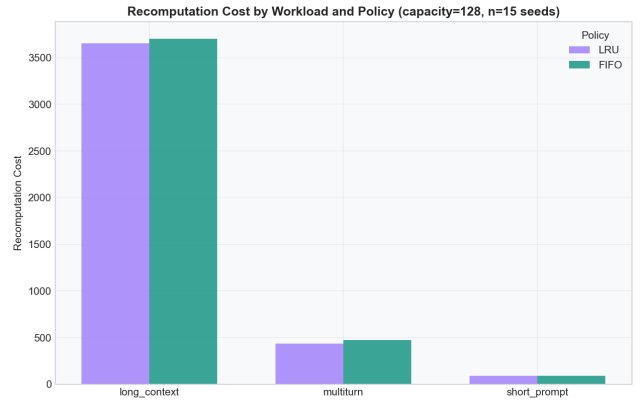


Figure 2: Recomputation cost across workload families for FIFO and LRU at capacity 128, averaged across 15 seeds.

5.2 Long-Context Scaling Behavior

The long-context workload is consistently the most challenging setting because its active working set is far larger than the cache size. As a result, both FIFO and LRU experience frequent evictions and misses, and absolute hit rates remain much lower than in short-context or multi-turn traces. Despite

this, the capacity sweep shows a clear memory-performance trend, where increasing cache capacity steadily improves hit rate and lowers recomputation cost.

An important pattern to note is that LRU improves more quickly than FIFO as capacity grows, indicating that temporal recency is still useful in long-context traces, even when locality is weaker. However, the overall hit-rate ceiling remains limited when the unique-block footprint is very large, suggesting that local eviction quality alone is not enough to fully address long-context pressure.

5.3 Shared Global Cache Optimization

A key limitation of isolated per-request caches is that they cannot share reusable blocks across concurrent requests. In practical LLM serving, requests often overlap in system prompts, prefixes, or repeated context structure. A shared-global cache enables cross-request reuse of these blocks, avoiding duplicate computation and improving effective memory utilization.

Figure 3 compares hit rate between local-isolated caching and shared-global caching under matched total memory budgets. Across both FIFO and LRU, shared-global caching substantially increases hit rate. FIFO benefits especially strongly in this setting because cross-request reuse helps compensate for weaker local eviction decisions.

Figure 4 shows the corresponding recomputation-cost impact. Shared-global caching produces large recomputation reductions by eliminating repeated recomputation of reusable blocks across requests. The reduction is substantial for both policies and is particularly pronounced for FIFO.

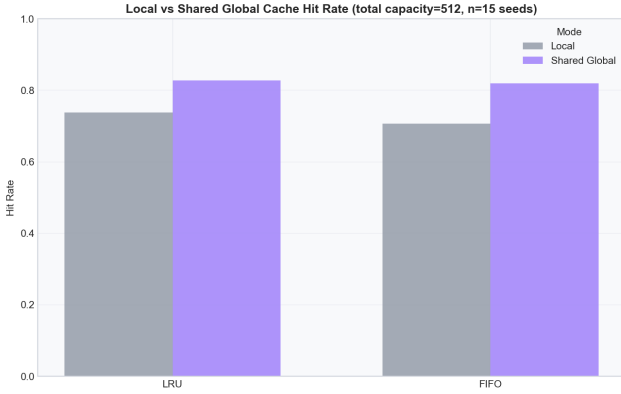


Figure 3: Hit rate comparison between isolated local caches and shared-global caching across FIFO and LRU policies.

To further characterize when shared-global caching is most effective, sensitivity experiments vary both shared-prefix reuse probability and concurrent request count. Figure 5 shows that hit-rate gains increase as overlap and concurrency increase, because more simultaneous requests create more opportunities for cross-request reuse.

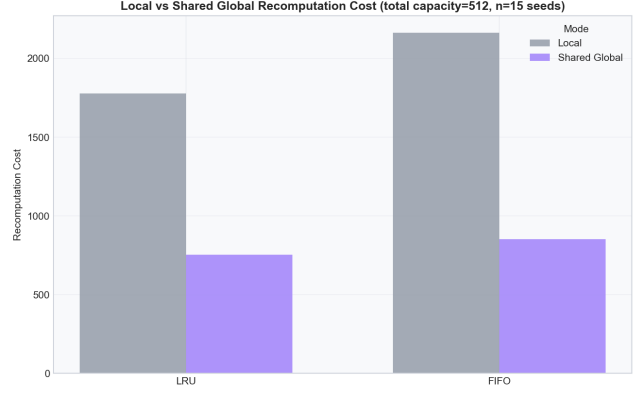


Figure 4: Recomputation cost comparison between isolated local caches and shared-global caching.

Figure 6 shows the matching recomputation trend, where recomputation reductions grow as shared reuse becomes more frequent and concurrency rises. Overall, LRU tends to extract more value from high-overlap scenarios due to recency-aware retention, while FIFO remains simpler and less adaptive.

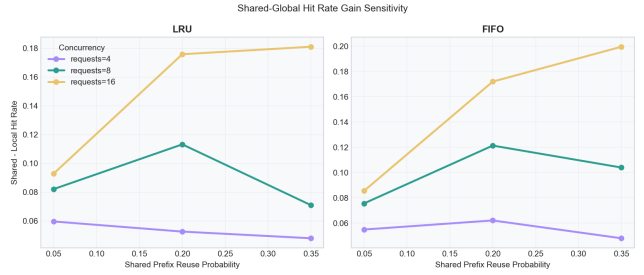


Figure 5: Sensitivity of shared-global hit-rate improvement under varying shared-prefix reuse probabilities and concurrency levels.

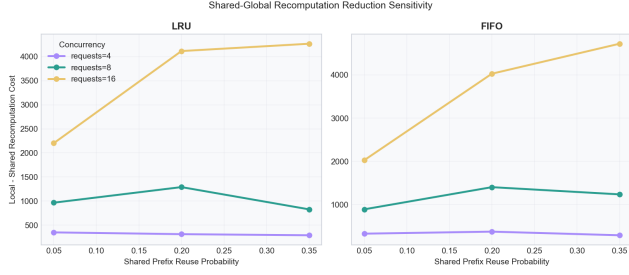


Figure 6: Sensitivity of recomputation reduction under varying shared-prefix reuse probabilities and concurrency levels.

5.4 Adaptive Eviction Strategy

While shared-global caching significantly improves cross-request reuse, workload behavior can still shift over time (for example, changes in locality, overlap, or access concentration). To address this, we implemented an adaptive eviction policy that adjusts itself based on observed cache pressure, instead of relying on a fixed eviction policy.

The adaptive strategy combines recency, reuse frequency, recomputation cost, and shared-prefix importance in a single scoring framework. It also applies pressure-aware admission, where if recent miss pressure rises, the policy becomes more selective about admitting new low-value blocks, reducing churn and protecting higher-value resident blocks. This allows the cache to react differently in stable regimes versus post-shift regimes.

Figure 7 reports overall hit rate across cache capacities for moderate and hard workload shifts. Here, the adaptive strategy consistently matches or slightly exceeds LRU and remains clearly stronger than FIFO, with the largest relative benefit appearing under harder shifts where workload non-stationarity is stronger.

Figure 8 reports overall recomputation cost, where adaptive consistently achieves the lowest recomputation cost across capacities and shift severities, indicating that cost-aware retention plus pressure-aware admission reduces expensive misses more effectively than static policies.

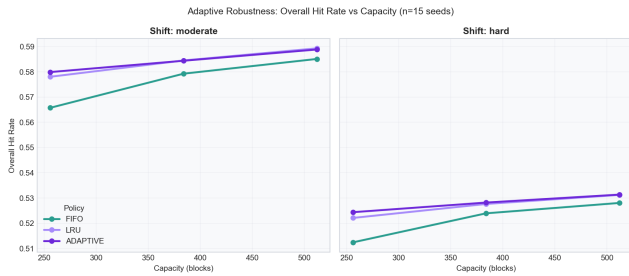


Figure 7: Overall hit rate versus cache capacity under moderate and hard workload shifts for FIFO, LRU, and adaptive eviction policies.

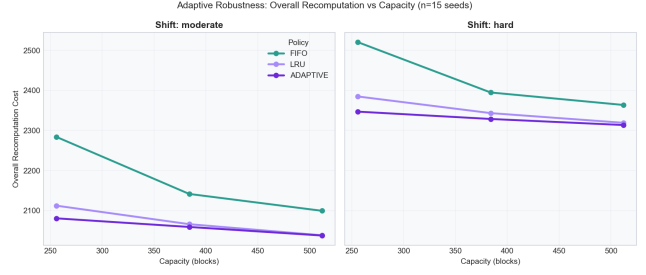


Figure 8: Overall recomputation cost versus cache capacity under workload shifts for FIFO, LRU, and adaptive eviction policies.

To examine transition robustness directly, Figure 9 breaks performance into phase-level results under a hard-shift setting. In Phase 1 (higher stability), policy differences are smaller because locality is more predictable. In Phase 2 (after the shift), adaptive maintains a modest but consistent advantage, suggesting that adjustments become more valuable when the access distribution changes.

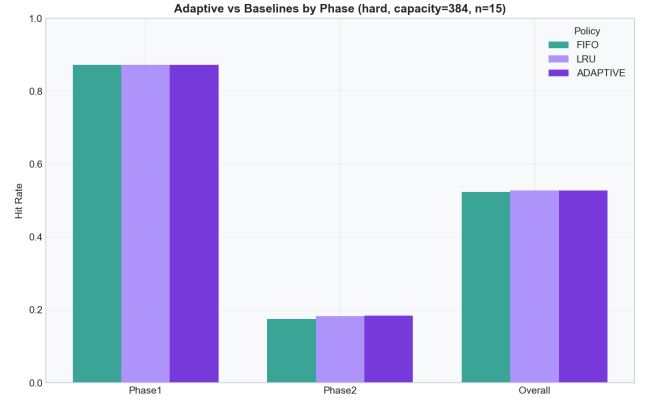


Figure 9: Phase-by-phase comparison of adaptive and baseline policies during a hard workload shift.

Overall, these results suggest that adaptive eviction provides incremental but reliable gains over static baselines. We observe that hit-rate improvements are generally not too significant, perhaps due to the fact that LRU is already a strong recency baseline and the shifted long-context working sets remain substantially larger than cache capacity, which limits absolute hit-rate even with better decisions, but recomputation savings are consistent and robustness is stronger under workload shifts.

6 Discussion

Our results suggest that KV-cache management should be treated as a systems-level problem rather than a low-level implementation detail. Policy behavior depends strongly on

workload structure, as we have shown that short-context traces are comparatively easy because the working set is small and highly reusable, long-context traces are hard because the active footprint exceeds cache capacity, and multi-turn traces lie between these extremes due to mixed prefix reuse and recent-turn locality.

A key takeaway we observe is that architectural changes can matter more than simply local policy changes. In our baseline, LRU generally outperforms FIFO, but the gain is modest compared with the improvement from moving to a shared-global cache. Isolated per-request caches duplicate reusable blocks and cannot exploit overlap across users, whereas shared-global caching enables cross-request reuse of common prefixes and reduces redundant recomputation, which is exactly the systems motivation highlighted by MOONCAKE.

We also find that adaptive eviction is most valuable in non-stationary settings. Under stable conditions, LRU is already a strong and competitive baseline. Under workload shifts and changing pressure, however, adaptive behavior provides more robust performance, especially through lower recomputation cost. In other words, adaptive policies may not always produce large hit-rate jumps, but they can still improve practical efficiency by protecting higher-value blocks when workload conditions change. Taken together, these findings suggest a practical design hierarchy for LLM serving systems, where we should first enable shared-global reuse to capture large system-level gains, and then apply adaptive control to improve robustness under dynamic workloads and memory pressure.

7 Limitations and Future Work

Our study is simulator-based rather than integrated into a full LLM serving stack. This gives us strong control and reproducibility, but it also means we do not directly measure end-to-end latency, throughput, tail latency, or generation quality. The workloads are synthetic and trace-driven, so they capture locality patterns by design but cannot represent all nuances of real production traffic or full attention dynamics.

A second limitation is scale. We report capacity in blocks and use relatively small-to-moderate cache sizes so experiments remain fast and laptop-friendly. At production-scale memory budgets, we expect parts of the curves to flatten as additional capacity yields diminishing returns. This does not invalidate our trends, but it does mean absolute values should be interpreted as comparative signals (policy A vs policy B under the same setup), not direct production forecasts. A natural next step is to sweep much larger capacities and fit saturation behavior explicitly.

Third, we assume fixed block granularity in the current simulator. Real systems may use variable block sizes, heterogeneous sequence chunking, or different allocation behavior across requests. If block sizes vary, both eviction and recomputation dynamics can change meaningfully (e.g., byte-efficient versus block-efficient decisions). Future work should extend

the simulator to size-aware accounting (bytes, not only block count) and evaluate fragmentation-aware eviction.

Fourth, our baseline policy set is intentionally narrow (FIFO and LRU) to establish clean, interpretable references for the 75% milestone before adding system-level and adaptive extensions. There are clearly other policies that could perform well here, including frequency-based approaches like LFU and hybrid designs such as S3-FIFO or ARC that combine recency and longer-term importance. We started with FIFO and LRU to isolate workload and architecture effects without introducing many interacting policy knobs. The goal was not to claim the globally best eviction policy, but to show that system design changes (especially shared-global caching) can produce larger gains than switching among simple local policies. We partially address this direction with the 125% adaptive policy, but a fuller comparison against additional hybrid baselines would strengthen the conclusions.

Looking forward, we see three future directions: (1) replaying traces collected from a real serving engine (e.g., vLLM [2]), (2) scaling experiments to larger cache budgets and concurrency levels, and (3) expanding policy comparisons to byte-aware and hybrid strategies while also studying fairness across short and long requests in shared-global scheduling.

8 Conclusion

This project studies KV-cache management for LLM inference through a trace-driven, block-level simulator. Across all experiments, we find that workload structure strongly shapes cache behavior. Short-context workloads are highly cache-friendly due to small working sets and strong locality, while long-context workloads remain challenging under limited capacity because the active footprint is much larger and reuse is weaker.

At the baseline level, LRU consistently outperforms FIFO by leveraging temporal locality, with the largest gains appearing in harder workloads. However, the largest overall improvement comes from system design: moving from isolated per-request caches to a shared-global cache enables cross-request reuse and substantially reduces redundant recomputation.

At the 125% stage, adaptive eviction provides additional robustness under workload shifts and changing memory pressure. Although hit-rate gains over strong static baselines are often modest, adaptive policies consistently reduce recomputation in dynamic settings by retaining higher-value blocks more effectively.

Real production systems use related ideas, as OpenAI and Anthropic have publicly described prompt caching support, where shared prefixes can be reused across requests to reduce latency and cost rather than recomputing from scratch [1, 3]. Similarly, while implementation details are often not fully public, modern LLM serving stacks likely rely on adaptive

cache-management behavior to handle changing workloads and memory constraints at runtime.

Overall, our findings suggest that KV-cache management should be treated as a storage and systems problem rather than only a local eviction-policy problem. As LLM serving workloads grow in sequence length, concurrency, and non-stationarity, shared-global and workload-aware cache designs are likely to be increasingly important for reducing recomputation, latency, and serving cost.

9 Extra Credit: Creative AI Usage

For this project, I experimented with a voice-driven, AI-assisted programming workflow by combining Wispr Flow for dictation with OpenAI Codex as a coding agent. Due to a hand injury during the semester (I cut my hand while slicing an avocado and had to get two stitches...), I relied heavily on speaking rather than typing to express implementation ideas, debugging goals, and system-design changes. Instead of manually writing every function or modification directly, I often described intended behavior in natural language and iteratively refined those instructions while Codex generated or edited portions of the simulator code.

This created a noticeably different interaction model compared to traditional software development. Rather than focusing primarily on syntax and line-by-line implementation, I had to do higher-level reasoning about workloads, cache behavior, experiment structure, and evaluation methodology. In practice, this meant I was translating systems ideas into precise verbal specifications that an AI agent could reliably interpret, and I found that vague or underspecified spoken instructions kept producing incorrect implementations, especially for adaptive-policy logic or workload-generation details, while clearer constraints and more structured prompts significantly improved the quality of generated code.

This ended up being useful for rapidly iterating on experiment infrastructure, plotting utilities, and repetitive implementation tasks. For example, AI assistance accelerated modifications to workload generators, cache-policy abstractions, and figure-generation pipelines, allowing more time to focus on interpreting results and designing experiments. At the same time, the process highlighted some limitations of our current agents, since generated implementations occasionally introduced logical inconsistencies, or they were optimized for superficially correct behavior rather than experimentally meaningful behavior, so a good bit of manual verification and debugging were still required.

Overall, this experiment demonstrated how combining voice interfaces with AI coding agents can shift programming away from purely manual syntax entry and toward higher-level problem specification and systems reasoning. While current tools still require careful oversight, I think this suggests that there might be a future development model in which programmers increasingly operate as designers and evaluators

of coding systems rather than being the complete implementation people.

Additional Reading

The following works influenced the broader design and systems perspective of this project, but are not directly cited in the main experimental discussion.

- Kwon, W., et al. *Efficient Memory Management for Large Language Model Serving with Paged Attention*. SOSP 2023.
- Megiddo, N., and Modha, D. S. *ARC: A Self-Tuning, Low Overhead Replacement Cache*. FAST 2003.
- Belady, L. A. *A Study of Replacement Algorithms for a Virtual-Storage Computer*. IBM Systems Journal, 1966.
- Vaswani, A., et al. *Attention Is All You Need*. NeurIPS 2017.
- Arpaci-Dusseau, R. H., and Arpaci-Dusseau, A. C. *Operating Systems: Three Easy Pieces*. 2018.
- Yang, J. *Cache in Storage Systems*. CS2640 Lecture Slides, Harvard University, 2026.

References

- [1] Anthropic. Prompt caching, 2024. <https://www.anthropic.com/news/prompt-caching>.
- [2] Woosuk Kwon et al. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS Symposium on Operating Systems Principles (SOSP)*, 2023.
- [3] OpenAI. Openai api documentation, 2024. <https://platform.openai.com/docs>.
- [4] Ruoyu Qin, Zheming Li, Weiran He, Jiale Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: Trading more storage for less computation — a kvcache-centric architecture for serving llm chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*, pages 155–170, Santa Clara, CA, February 2025. USENIX Association.